Padrão Arquitetural — PB DECOM / Jobs

> **Objetivo:** Documentar o padrão de arquitetura adotado para servir de referência em novos projetos do mesmo tipo.
> **Stack de referência:** .NET (C#) 9 · ASP.NET Core Web API · SQL Server

---

1. Contexto e Motivação

Este padrão foi definido para projetos que envolvem **processamento batch**, com execução agendada via Autosys. Ele é indicado quando o projeto apresenta **uma ou mais** das seguintes características:

Execução agendada (Job) O processamento tem um ciclo de vida bem definido: começa, executa uma sequência de etapas e termina. Não é um serviço continuamente ativo.

Leitura de arquivos externos O sistema recebe arquivos gerados por sistemas externos (legados, parceiros, bolsas, etc.) como fonte de dados. Esses arquivos chegam no sistema de arquivos e precisam ser lidos, validados e parseados antes de qualquer processamento.

Dados de múltiplos bancos O sistema não tem um banco próprio exclusivo — ele lê e escreve em bancos existentes, cada um com seu propósito específico e isso exige gerenciamento de múltiplas conexões.

Regras de negócio complexas A transformação dos dados não é trivial. Existem fórmulas, condições, casos especiais e variações por tipo de entidade que precisam estar expressas de forma clara, testável e isolada — não espalhadas em queries SQL ou misturadas com código de infraestrutura.

A arquitetura foi desenhada para garantir **separação clara de responsabilidades**, **testabilidade** e **facilidade de manutenção e evolução**.

---

2. Visão Geral da Solução

A solução é composta por **dois processos distintos e complementares**:

Componente	Tipo	Responsabilidade
Job	Console App (.NET)	Entry point do processamento. Configura DI, resolve Use Cases e os executa em sequência. Não contém lógica de negócio.
API	ASP.NET Core Web API	Única camada com acesso ao banco de dados. Todo acesso a dados — seja pelo Job ou por sistemas externos — passa obrigatoriamente por ela.

Por que separar Job e API?

A separação existe por algumas razões:

- **O Job não acessa o banco diretamente.** Todo acesso a dados é feito via chamadas HTTP para a API. Isso centraliza as regras de acesso, facilita auditoria e permite que outros sistemas consumam os mesmos endpoints se necessário.
- **A API não executa o processamento.** Ela apenas expõe dados. Isso evita que a API acumule responsabilidades de orquestração e mantenha seus endpoints simples e previsíveis.
- **Consultas externas** podem ser feitos via API sem precisar disparar o Job inteiro.

---

3. Arquitetura em Camadas (Clean Architecture)

O projeto segue os princípios da **Clean Architecture**, organizando o código em quatro camadas com dependências sempre apontando para dentro — camadas externas dependem das internas, nunca o contrário.

```code
┌──────────────────────────────────┐
│             API / Job            │  ← Entry points (mais externo)
├──────────────────────────────────┤
│          Infrastructure          │  ← Implementações concretas
├──────────────────────────────────┤
│           Application            │  ← Use Cases + contratos (interfaces)
├──────────────────────────────────┤
│             Domain               │  ← Regras de negócio puras (mais interno)
└──────────────────────────────────┘
```

A regra fundamental é: **Domain não conhece ninguém. Application conhece só o Domain. Infrastructure e API conhecem todos, mas são conhecidas por ninguém.**

---

4. Camadas em Detalhe

4.1 Domain

O que é: O núcleo da aplicação. Contém exclusivamente lógica de negócio pura, sem nenhuma dependência externa — sem banco, sem HTTP, sem framework.

O que fica aqui:

- Regras de negócio expressas como classes estáticas
- Enums de domínio
- Validadção de domínio (que lançam exceções de negócio)
- Exceções de domínio customizadas

Por que assim: Isolar as regras de negócio garante que elas possam ser testadas unitariamente sem nenhum mock de infraestrutura. Também facilita que a mesma regra seja reutilizada em contextos diferentes.

Decisão de design: Regras de negócio que aparecem em múltiplos lugares do código (ex: inversão de sinal, conversão de taxa, cálculo de PU) devem viver no Domain como classes estáticas puras, não espalhadas nos Use Cases.

4.2 Application

O que é: A camada de orquestração. Contém os **Use Cases** — cada um responsável por uma operação de negócio bem delimitada — e define as **interfaces** que a infraestrutura deve implementar.

O que fica aqui:

- **Use Cases:** Classes com um método principal (`ExecutarAsync`) que orquestram o fluxo de uma operação
- **Interfaces de repositório:** Contratos de acesso a dados (`IRepository`)
- **Interfaces de clientes HTTP:** Contratos para chamadas à API (`IApiClient`)
- **Interfaces de serviços:** Contratos para serviços de suporte (log, leitura de arquivo, etc.)
- **DTOs, Requests e Responses:** Objetos de transferência de dados entre camadas

Por que assim: A Application sabe *o que* precisa ser feito, mas não sabe *como* — ela depende de abstrações.

Decisão de design — Use Cases granulares: Cada Use Case deve ter uma única responsabilidade. Use Cases complexos devem ser decompostos em sub-Use Cases menores e orquestrados por um Use Case pai. Isso facilita testes, reuso e manutenção isolada de cada etapa.

Decisão de design — sem acesso direto ao banco: A Application nunca instancia conexões ou faz queries. Ela depende de interfaces que serão implementadas na Infrastructure.

4.3 Infrastructure

O que é: A camada de implementação. Aqui vivem todas as implementações concretas dos contratos definidos na Application. É a única camada que "sabe" como se conectar ao banco, como fazer uma chamada HTTP e como ler um arquivo do disco.

A Infrastructure é dividida em subcamadas, cada uma com escopo bem delimitado:

- **Infrastructure.SqlServer** — gerenciamento de conexões com múltiplos bancos SQL Server, descriptografia de credenciais e repositórios usando Dapper.
- **Infrastructure.Http** — criação e configuração dos `HttpClient` e implementação dos API Clients que o Job usa para se comunicar com a API.
- **Infrastructure.FileSystem** — tudo relacionado a arquivos: leitura, validação de existência, parsing de conteúdo e extração de dados de arquivos externos recebidos de sistemas legados ou parceiros.
- **Infrastructure.Security** — serviços de criptografia e descriptografia.
- **Infrastructure.Logging** — implementação dos serviços de log, desacoplando os Use Cases do mecanismo concreto de escrita de logs.
- **Infrastructure.Email** — envio de e-mails via configuração SMTP, com integração à biblioteca interna `BSG.Notifications`.

Cada subcamada pode ser registrada independentemente no container de DI.

Como a conexão com banco funciona na prática:

A Infrastructure resolve o problema de múltiplos bancos com três componentes em cadeia:

```code
Repositório
    └── SqlRepositoryBase
            └── ConnectionService
                    └── ConfigurationHelper
                            └── EncryptionService
```

O **repositório** pede uma conexão pelo contexto (`CreateMoedasConnectionAsync()` , `CreateMercadoConnectionAsync()` , etc.) sem saber nada sobre strings ou senhas. O **`ConnectionService`** sabe qual connection string do `appsettings.json` corresponde a cada banco e delega a descriptografia para o **`ConfigurationHelper`** . O **`ConfigurationHelper`** detecta se é ambiente local (`Integrated Security`) — e devolve a string como está — ou se é ambiente de UAT/Produção — e aciona o **`EncryptionService`** para descriptografar a senha antes de montar a string final.

Isso garante que: (1) os repositórios nunca lidam com segurança, (2) a lógica de descriptografia fica em um único lugar, e (3) o mesmo código funciona em local e em produção sem alteração.

Como os API Clients funcionam na prática:

Cada domínio tem um client próprio que implementa a interface definida na Application. Internamente, todos delegam para um `IApiClient` genérico — que encapsula o `HttpClient` e os métodos `GetFromJsonAsync` , `PostAsync` , etc. O Use Case injeta a interface do client de domínio (`IXxxApiClient`) e chama métodos com nomes de negócio, sem saber que por baixo existe HTTP.

```code
Use Case → IXxxApiClient (interface)
               └── XxxApiClient (implementação na Infrastructure)
                       └── IApiClient (genérico)
                               └── HttpClient (.NET)
```

Isso significa que, para testar um Use Case, basta mockar a interface — nenhum servidor HTTP precisa estar rodando.

Decisão de design — `SqlRepositoryBase` : Todos os repositórios herdam desta classe base. Ela recebe o `ConnectionService` via construtor e expõe métodos protegidos de criação de conexão por banco. Nenhum repositório instancia `SqlConnection` diretamente com uma string hardcoded. Isso centraliza a gestão de conexões e garante consistência.

Decisão de design — um client por domínio: Em vez de um único client genérico chamando qualquer endpoint, cada domínio tem sua própria interface e implementação. Isso torna as dependências dos Use Cases explícitas — se um Use Case precisa de dados de moedas e de feriados, seus construtores deixam isso claro.

Decisão de design — Infrastructure não contém lógica de negócio: Repositórios executam queries e mapeiam resultados. Clients fazem chamadas HTTP e desserializam respostas. Nenhum deles decide *o que fazer* com os dados — essa decisão pertence aos Use Cases.

4.4 API (.NET 9)

O que é: A camada de entrada para acesso a dados. É a **única porta de acesso ao banco** — tanto o Job quanto sistemas externos passam por ela.

O que fica aqui:

- Controllers REST organizados por domínio/contexto
- Endpoint de health check (`/health`)
- Configuração do pipeline HTTP (middlewares, roteamento)

Por que assim: Centralizar o acesso ao banco na API garante um ponto único de controle sobre leitura e escrita de dados. Qualquer regra de acesso, auditoria ou throttling pode ser aplicada em um único lugar.

Decisão de design — Controllers delegam para repositórios diretamente: Os controllers da API não contêm lógica. Eles recebem a requisição, chamam o repositório e retornam o resultado.

Decisão de design — API não conhece o Job: A API não referencia nenhuma classe do Job. O fluxo de dados é unidirecional: Job → API → Banco.

4.5 Job (Console App / Entry Point)

O que é: O ponto de entrada do processamento batch. É responsável por configurar a injeção de dependência e chamar os Use Cases na ordem correta.

O que fica aqui:

- `Program.cs`: configuração de DI e sequência de execução
- Nada mais.

Por que assim: Manter o `Program.cs` simples é intencional. Se ele crescer com lógica condicional e orquestração complexa, é sinal de que essa lógica deveria estar em um Use Case.

Decisão de design — DI modularizada por camada: O registro de serviços é feito por métodos de extensão separados por camada (`AddHttpInfrastructure`, `AddSqlServerInfrastructure`, `AddMercadosFuturosApplication`, etc.).

---

5. Fluxo de Dados

O Job é disparado externamente pelo **Autosys**

```code
     Autosys
        │
        │  dispara o Job no horário configurado
        ▼
   Job (Console App)
        │
        │  Use Cases orquestram o processamento
        │  Fontes de dados: API (via HTTP) + FileSystem (arquivos externos)
        │
        ▼
       API  ◄──── Sistemas externos
        │
        │  Controllers chamam Repositórios
        │
        ▼
   SQL Server (múltiplos bancos)
```

---

6. Injeção de Dependência

Toda a composição do sistema é feita via DI do .NET (`IServiceCollection`). Nenhuma classe instancia suas dependências diretamente — todas são injetadas via construtor.

A organização dos registros segue o mesmo padrão das camadas:

```csharp
// Infrastructure
services.AddHttpInfrastructure(configuration);      // HttpClient + API Clients
services.AddSqlServerInfrastructure();              // Repositórios + ConnectionService
services.AddSecurityInfrastructure();               // EncryptionService
services.AddFileSystemInfrastructure(configuration); // LeitorArquivoService

// Application
services.AddNomeDoContextoApplication();            // Use Cases + ConfigurationHelper
```

Decisão de design: Cada camada expõe um método de extensão próprio para registro. O `Program.cs` não sabe quais classes concretas estão sendo registradas — só sabe que a camada está configurada.

---

7. Configuração e Segurança

As configurações ficam no `appsettings.json`, separadas por contexto:

- **Connection Strings:** Uma por banco, com suporte a dois modos — `Integrated Security` e usuário/senha criptografada (ambientes de UAT/Produção)
- **Credenciais criptografadas:** Senhas são armazenadas criptografadas no `appsettings.json` e descriptografadas em runtime pelo `EncryptionService` (RFC 2898).

- **Parâmetros de negócio:** Valores parametrizados ficam no `appsettings.json` — não hardcoded no código.

---

8. Logging

Projetos que seguem este padrão tendem a ter necessidades distintas de log que não se encaixam em um único canal. É comum separar, por exemplo, logs operacionais (acompanhamento do processamento em tempo real) de logs de auditoria (rastreabilidade persistida do que foi executado). A quantidade e o tipo de canais varia por projeto.

Decisão de design: Independente de quantos ou quais canais o projeto utilizar, todos devem ser definidos como interfaces na camada Application e injetados nos Use Cases via DI.

---

9. Tratamento de Erros e Casos Especiais

Casos especiais de negócio (ex: feriados, ausência de dados, datas alternativas) são tratados **dentro dos Use Cases**, com lógica explícita e logs descritivos. A regra geral é:

- **Erro esperado de negócio** → loga, toma decisão alternativa ou aborta com mensagem clara
- **Erro inesperado** → propaga a exceção para o `Program.cs`, que encerra o processo

Não se usa captura genérica de exceções para esconder problemas. Cada Use Case é responsável por tratar apenas os erros que conhece.

---

10. Checklist para Novos Projetos

Ao iniciar um novo projeto seguindo este padrão, verificar:

- [ ] As 5 camadas estão criadas: `Domain`, `Application`, `Infrastructure`, `Api`, `Jobs`
- [ ] Domain não referencia nenhuma outra camada
- [ ] Application depende apenas de Domain (via interfaces, nunca de implementações concretas)
- [ ] Infrastructure implementa as interfaces da Application
- [ ] A API é a única camada com acesso ao banco
- [ ] O Job acessa dados exclusivamente via API Clients HTTP
- [ ] Cada Use Case tem responsabilidade única e bem definida
- [ ] O `Program.cs` contém apenas configuração de DI e chamada dos Use Cases
- [ ] Senhas e credenciais nunca estão em texto fora do appsettings
- [ ] Os canais de log estão implementados e injetados como interfaces
- [ ] Parâmetros de negócio estão no `appsettings.json`, não hardcoded